2026 MCM Senior Group Problem B
Scheduling a GPU Cluster

Background and Data

Graphics processing units (GPUs) are widely used for modern artificial intelligence and scientific computing. A large computing center usually owns many GPU servers. Different servers may contain different GPU models and different numbers of CPU cores. Meanwhile, computing jobs arrive over time and request different amounts of GPU and CPU resources.

This problem is inspired by an open-source GPU-cluster trace released in the Alibaba Cluster-Data repository. For this competition, we provide a cleaned version containing only the first three months of job submissions. We remove the original job-priority and organization information, and we remove jobs with fractional GPU requests so that every GPU request in the supplied data is an integer. However, CPU requests are measured in vCPUs and may be fractional.

You will receive two data files. The first file, `node_info_df.csv`, describes the available servers (called *nodes*). The second file, `job_info_clean.csv`, describes the jobs that request computing resources.

Node information. Each row of `node_info_df.csv` represents one GPU server.

Header	Meaning
`node_name`	An anonymized identifier of the server.
`gpu_model`	The GPU model installed on the server, such as A10 or A100-SXM4-80GB.
`gpu_capacity_num`	The number of physical GPUs on this server.
`cpu_num`	The CPU capacity of this server, measured in virtual CPU cores (vCPUs).

Job information. Each row of `job_info_clean.csv` represents one submitted computing job. For this problem, a job consists of several identical *workers*.

Header	Meaning
`job_name`	A unique anonymized identifier of the job.
`gpu_model`	The GPU model requested by this job. A worker can only be placed on a node with the matching GPU model.
`gpu_request`	The number of GPUs required by each worker. In the supplied data this is a positive integer.
`cpu_request`	The number of vCPUs required by each worker. It may be fractional.
`worker_num`	The number of workers in the job. All workers of a job must start at the same time, but different workers may be placed on different compatible nodes.
`submit_time`	The time when the job is submitted, measured in seconds from the beginning of the trace.
`duration`	The running time of the job, measured in seconds. Once the job starts, we assume it occupies its allocated resources continuously for this amount of time. Assume the duration is known when a job is submitted.

For example, if a job has `worker_num`$= 4$, `gpu_request`$= 2$, and `cpu_request`$= 16$, then it has four workers, each requiring 2 GPUs and 16 vCPUs. Hence the whole job requests 8 GPUs and 64 vCPUs in total, although these resources may be distributed among several compatible nodes. In this problem we do not consider job preemption: once a job starts, it runs until completion.

Questions

(1) **Understand the data.** Can we analyze the data and find some patterns? For example, it may help a manager understand what kinds of jobs arrive, which GPU models are most requested, how job sizes and running times vary, whether demand changes substantially over time, and whether a small number of very large jobs account for a large fraction of the requested computing work. You can also compare the demand for different GPU models with the available server capacity and identify possible resource imbalances.

Hints: Plots, cumulative distributions, daily or weekly time series, and summary statistics may be useful. You are encouraged to explain why the statistics you choose are informative rather than simply producing many figures.

(2) **Design a scheduling algorithm.** Suppose that all jobs must be scheduled using the servers in `node_info_df.csv`. When a job arrives, it may start immediately or wait in a queue. A job can start only when all of its workers can be placed simultaneously. Workers belonging to the same job may be assigned to different nodes, but each worker must receive its requested CPU and GPU resources on the same node. At every time the total numbers of GPUs and vCPUs assigned on any node cannot exceed that node's `gpu_capacity_num` and `cpu_num`. Once a job starts, it runs for its given `duration` and cannot be preempted. Design a scheduling and placement algorithm for this system. Assume that all servers are initially idle at time zero.

Hints:

(a) A scheduling decision has two parts: deciding *which* waiting jobs should start and deciding *where* their workers should be placed. A simple idea is to use the first-come-first-served scheduling. Is it always the best? How about making the shortest job first, or using backfilling, or gradually increasing priority to jobs that have waited for a long time?

(b) Possible criteria for a good scheduling rule include average and median waiting time, long waiting times such as the 95th percentile or maximum waiting time, GPU and CPU allocation rates, the number of jobs completed within a fixed time horizon, fairness among jobs, and the computational cost of the scheduling algorithm itself. There may be trade-offs between these criteria, so you should clearly state what you are trying to improve and why.

(3) **Reduce imbalance across GPU models.** You may notice that different GPU models experience very different demand. In particular, A10 jobs may face much heavier congestion than jobs requesting higher-end A100 or A800 GPUs, leaving some expensive computing capacity under-used while another GPU pool has a long queue. Propose and evaluate one or more methods for reducing such an imbalance.

Hints: There is no single expected answer. Some hints include:

(a) Technical approaches: Allowing some jobs to run on alternative compatible GPU models, or dividing selected high-end GPUs into smaller virtual GPU units for jobs that do not require a whole physical card. Technologies such as NVIDIA Multi-Instance GPU (MIG) provide one real-world motivation for such partitioning.

(b) Financial approaches: For example, an under-used GPU type may receive a discount, encouraging flexible users to switch. However, the supplied trace does not contain actual customer prices or willingness-to-pay information. If you introduce pricing or cross-GPU substitution, you must therefore state and justify additional assumptions, or use some public pricing information.

Instructions

- **Use of AI Tools**: Teams are allowed to use AI tools, including ChatGPT, DeepSeek, and other AI-assisted software. However, all AI use must be clearly disclosed. In the report, please include a brief paragraph on *Use of AI Tools* at the end of your submission, including the tools used and their purposes. All AI-generated content, calculations, code, and references must be carefully checked for correctness and plagiarism. Teams remain fully responsible for their submitted work.

- **Submission**: Each team should submit a short report (**no more than 10 A4 pages, font size no smaller than 11pt**) in English explaining your answers and ideas. Please include the most important figures, tables, mathematical models, and numerical results in the report.

  Please list the names of all members in your report. Each team only has to ask the Team Leader to submit the files through the CMT system before October 20, 2026, 23:59 Beijing Time:

  `https://cmt3.research.microsoft.com/HKUSTGZMCM2026`

  - Main submission file: The report, in doc, docx, or pdf format.
  - If there are any source codes used, please submit them as **Supplementary Materials** in a **ZIP** file no larger than 50 MB.

- **Copyright:** Each team retains the copyright of their work. We will only use your submissions to decide the winner.

- **Data Source and Acknowledgment**: The data used in this problem are adapted from the publicly released Alibaba Cluster Trace. The competition organizers have preprocessed the original trace for educational purposes. We gratefully acknowledge Alibaba Group and the authors of the GFS project[1] for making the dataset publicly available. Participants who refer to the original dataset or paper should cite the source appropriately.

- **Academic Integrity:** This competition, like all other competitions, functions best when the participants treat one another with honesty, fairness, respect, and trust. The organizers hold all participants of this competition to high standards of scholarship and integrity.

  To accomplish the mission of providing an optimal educational environment, we promote the assumption of personal responsibility and integrity and prohibit all forms of academic dishonesty and misconduct. Conduct, without regard to motive, that violates the academic integrity and ethical standards of the University cannot be tolerated. Violations of the standards will result in serious consequences and disciplinary action according to the regulations by the University.

[1] `https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2026-spot-gpu`